

Does Random Mean Secure?

Randomness, Entropy and Perfect Secrecy

tdvnm

Probability and Statistics — project proposal

September 16, 2026

What this talk covers

Introduction

Understanding Randomness

Statistical Tests for Randomness

Human Randomness vs Computer Randomness

Random Numbers vs Pseudorandom Numbers

Introducing Encryption

Statistical Weakness of Simple Ciphers

Research direction

Two meanings of “random”

1. A sequence *looks* random. Half zeros, half ones, runs of the right length, nothing repeats. (statistics)
2. A sequence *is* unpredictable. Knowing everything else, you still cannot guess the next bit. (cryptography)

Two meanings of “random”

1. A sequence *looks* random. Half zeros, half ones, runs of the right length, nothing repeats. (statistics)
2. A sequence *is* unpredictable. Knowing everything else, you still cannot guess the next bit. (cryptography)

These come apart in both directions.

- A sequence can pass every test and be fully predictable.
- A ciphertext can look like noise and still leak the message.

Two meanings of “random”

1. A sequence *looks* random. Half zeros, half ones, runs of the right length, nothing repeats. (statistics)
2. A sequence *is* unpredictable. Knowing everything else, you still cannot guess the next bit. (cryptography)

These come apart in both directions.

- A sequence can pass every test and be fully predictable.
- A ciphertext can look like noise and still leak the message.

Cryptography needs the second kind. Statistics can only check the first. The project is about the gap between them.

Problem statement

Main question

If the randomness used to make an encryption key gets worse, how much more does the ciphertext reveal about the message?

Two smaller questions sit on either side of it.

1. How do you test, with statistics, whether a sequence behaves randomly? What do those tests miss?
2. If a sequence passes the tests, is it safe to use as a key?

Problem statement

Main question

If the randomness used to make an encryption key gets worse, how much more does the ciphertext reveal about the message?

Two smaller questions sit on either side of it.

1. How do you test, with statistics, whether a sequence behaves randomly? What do those tests miss?
2. If a sequence passes the tests, is it safe to use as a key?

I expect the answer to the last one is *no*. The point of the project is to show why, with a derivation and a simulation for each claim, using the tools from this course.

Why anyone should care

Every key, nonce and session token is a number an attacker must not guess. When the randomness behind it is bad, the cipher does not matter.

- **Debian OpenSSL, 2006–08.** A patch cut the seeding down to the process ID. Every key made in two years came from a pool of 32,767.
- **Sony PlayStation 3, 2010.** The same “random” number was reused in every signature. That gave away the private key.

Why anyone should care

Every key, nonce and session token is a number an attacker must not guess. When the randomness behind it is bad, the cipher does not matter.

- **Debian OpenSSL, 2006–08.** A patch cut the seeding down to the process ID. Every key made in two years came from a pool of 32,767.
- **Sony PlayStation 3, 2010.** The same “random” number was reused in every signature. That gave away the private key.

Both systems used sound ciphers. Nothing in the output gave the failure away. The keys and signatures looked fine. The failure was in the probability, not the algorithm.

That is a failure you can only see if you model the randomness itself. This project does that on systems small enough to work out by hand.

Why small systems

I do not use AES or RSA. I use coin flips, a linear congruential generator, a Caesar cipher, XOR and the one-time pad.

- Every one of them has a joint distribution I can write down.
- So every simulation has an exact answer to be checked against.
- The lessons transfer: Shannon's definition of secrecy does not care how big the cipher is.

The plan is always the same. Derive the number. Simulate the system. See that they agree.

One project, two models

Model I: a bit source

Is this sequence random?

- A hypothesis test.
- Null: the bits are i.i.d. fair coins.
- Alternatives: a biased coin, a person, a deterministic generator.
- Output: a p -value per test.

Model II: a cipher

What does the ciphertext give away?

- A joint distribution of three random variables M, K, C .
- Secrecy is a statement about $\mathbb{P}(M = m \mid C = c)$.
- Leakage is a number, $I(M; C)$.
- Output: an exact curve and a simulated one.

Model I is the first half of the project. Model II is the second. The last part joins them: a source that passes Model I and fails as a key in Model II.

The chain I want to follow

randomness $\rightarrow$ uncertainty $\rightarrow$ information $\rightarrow$ secrecy

Four studies, one per link. Each has a claim, a derivation, a simulation, and a way to be wrong.

1. **Test the sources.** Which tests catch which failures?
2. **Measure the ciphers.** How much does each one leak?
3. **Degrade the key.** The main experiment: leakage as a function of key entropy.
4. **Break a good-looking generator.** Pass every test, still predictable.

Model I: the random variables

A sequence is $B_1, B_2, \dots, B_n$ with each $B_i \in \{0, 1\}$.

Null hypothesis H_0 (an ideal source)

$B_1, \dots, B_n$ i.i.d. $\text{Bern}(\frac{1}{2})$. Every one of the 2^n sequences has probability 2^{-n} .

Model I: the random variables

A sequence is $B_1, B_2, \dots, B_n$ with each $B_i \in \{0, 1\}$.

Null hypothesis H_0 (an ideal source)

$B_1, \dots, B_n$ i.i.d. $\text{Bern}(\frac{1}{2})$. Every one of the 2^n sequences has probability 2^{-n} .

Alternatives, each a distribution I can simulate from:

- **Biased coin.** B_i i.i.d. $\text{Bern}(p)$, $p \neq \frac{1}{2}$.
- **A person.** A two-state Markov chain on $\{0, 1\}$ with switching probability $s = \mathbb{P}(B_{i+1} \neq B_i \mid B_i)$. People who try to “be random” have $s > \frac{1}{2}$: they avoid long runs.
- **A generator.** $X_{i+1} = (aX_i + c) \bmod m$, output one bit of X_i . Not random at all: given X_0 the sequence is fixed.

Model I: the random variables

A sequence is $B_1, B_2, \dots, B_n$ with each $B_i \in \{0, 1\}$.

Null hypothesis H_0 (an ideal source)

$B_1, \dots, B_n$ i.i.d. $\text{Bern}(\frac{1}{2})$. Every one of the 2^n sequences has probability 2^{-n} .

Alternatives, each a distribution I can simulate from:

- **Biased coin.** B_i i.i.d. $\text{Bern}(p)$, $p \neq \frac{1}{2}$.
- **A person.** A two-state Markov chain on $\{0, 1\}$ with switching probability $s = \mathbb{P}(B_{i+1} \neq B_i \mid B_i)$. People who try to “be random” have $s > \frac{1}{2}$: they avoid long runs.
- **A generator.** $X_{i+1} = (aX_i + c) \bmod m$, output one bit of X_i . Not random at all: given X_0 the sequence is fixed.

Each test picks a statistic T , works out the law of T under H_0 , and reports the p -value $\mathbb{P}(T \text{ at least as extreme as } t_{\text{obs}} \mid H_0)$.

Model I: entropy of a source

Not a test statistic but a measure of how unpredictable the source is.

$$H(B) = - \sum_{b \in \{0,1\}} \mathbb{P}(B = b) \log_2 \mathbb{P}(B = b) = h(p) = -p \log_2 p - (1-p) \log_2 (1-p).$$

It is 1 bit only when $p = \frac{1}{2}$. For $p = 0.9$ it is 0.47 bits. I estimate it by plugging in the observed frequency.

Model I: entropy of a source

Not a test statistic but a measure of how unpredictable the source is.

$$H(B) = - \sum_{b \in \{0,1\}} \mathbb{P}(B = b) \log_2 \mathbb{P}(B = b) = h(p) = -p \log_2 p - (1-p) \log_2 (1-p).$$

It is 1 bit only when $p = \frac{1}{2}$. For $p = 0.9$ it is 0.47 bits. I estimate it by plugging in the observed frequency.

For a sequence the question is the conditional one: how much is left to guess about B_{n+1} once $B_1, \dots, B_n$ are known, $H(B_{n+1} \mid B_1, \dots, B_n)$. For the ideal source it is 1 bit. For a generator it is 0 once the state is known. Every test in the next section looks at the output; none of them can see this number.

Model I: frequency and runs

Frequency. $X = \sum_{i=1}^n B_i$, the number of ones. Under H_0

$$X \sim \text{Bin}(n, \tfrac{1}{2}), \quad \mathbb{E}[X] = \tfrac{n}{2}, \quad \text{Var}(X) = \tfrac{n}{4},$$

and the CLT gives $(X - n/2)/\sqrt{n/4} \approx \mathcal{N}(0, 1)$. Catches the biased coin. Misses the person.

Model I: frequency and runs

Frequency. $X = \sum_{i=1}^n B_i$, the number of ones. Under H_0

$$X \sim \text{Bin}(n, \tfrac{1}{2}), \quad \mathbb{E}[X] = \tfrac{n}{2}, \quad \text{Var}(X) = \tfrac{n}{4},$$

and the CLT gives $(X - n/2)/\sqrt{n/4} \approx \mathcal{N}(0, 1)$. Catches the biased coin. Misses the person.

Runs. Let $D_i = B_i \oplus B_{i+1}$ mark a switch, and $R = 1 + \sum_{i=1}^{n-1} D_i$ count the runs. The map $(B_1, \dots, B_n) \mapsto (B_1, D_1, \dots, D_{n-1})$ is a bijection of $\{0, 1\}^n$, so under H_0 the D_i are again i.i.d. $\text{Bern}(\frac{1}{2})$ and

$$R - 1 \sim \text{Bin}(n - 1, \tfrac{1}{2}).$$

For the person, D_i are i.i.d. $\text{Bern}(s)$ instead, so $R - 1 \sim \text{Bin}(n - 1, s)$. Same test statistic, different law, and the test sees it.

Model I: summary of the random variables

Variable	Definition	Law under H_0	Test
B_i	one bit	$\text{Bern}(\frac{1}{2})$, independent	—
X	$\sum_{i=1}^n B_i$	$\text{Bin}(n, \frac{1}{2})$	frequency
R	$1 + \sum_{i=1}^{n-1} 1\{B_i \neq B_{i+1}\}$	$R - 1 \sim \text{Bin}(n - 1, \frac{1}{2})$	runs
$H(B)$	$h(p)$	$= 1 \text{ bit iff } p = \frac{1}{2}$	entropy
P	p -value of any test	$\text{Unif}(0, 1)$	calibration

The last row is how I check the tests themselves. If a test is right, its p -value on a truly fair source is uniform on $[0, 1]$. I will run each test on thousands of fair sequences and look at the histogram before I trust it on anything else.

Study 1: test the sources

Sources. Fair coin, biased coin ($p = 0.6$), human sequences (collected from classmates, $n \approx 100$ each), an LCG, and the OS generator.

Method. 1000 sequences of $n = 1000$ bits per source (fewer for people). Run all three tests on each. Record p -values and the plug-in entropy. Fit $\hat{s}$ (switching probability) to the human data.

Claims.

- The frequency test catches the biased coin and nothing else.
- The runs test catches people: $\hat{s} > \frac{1}{2}$ and R too large.
- The LCG and the OS generator pass all three.

Wrong if the human sequences look like fair coins under every test, or if the LCG fails a test. Either would change Study 4.

Study 4: break a good-looking generator

Back to the LCG from Study 1. It passed every test. Is it random?

Claim. For a real source $H(B_{n+1} \mid B_1, \dots, B_n) = 1$ bit: the past says nothing about the next bit. For the LCG it is 0 once the state is known, and the state can be recovered from a few outputs.

Method.

- Use a small modulus m . Try every possible state X_0 and keep the ones that match the observed bits. Count how many outputs it takes until only one is left.
- Use the LCG as the key stream of the cipher in the next section. Show that an attacker who sees a short prefix of the plaintext reads the rest.

This is the slide the title is about. The LCG's output has entropy 1 bit per bit as far as the tests can tell, and 0 bits per bit to someone who knows the state. Statistical randomness is a property of the output. Security is a property of what the attacker knows. Real cryptographic generators are built so that recovering the state is computationally infeasible; the LCG is not.

Model II: the random variables

Three random variables on $\{0, 1\}^n$:

M (message), K (key), $C = M \oplus K$ (ciphertext).

Assumption

K is independent of M . The attacker sees C , knows the cipher, knows $\mathbb{P}(M = m)$ and $\mathbb{P}(K = k)$, and does not know K .

Model II: the random variables

Three random variables on $\{0, 1\}^n$:

$$M \text{ (message),} \quad K \text{ (key),} \quad C = M \oplus K \text{ (ciphertext).}$$

Assumption

K is independent of M . The attacker sees C , knows the cipher, knows $\mathbb{P}(M = m)$ and $\mathbb{P}(K = k)$, and does not know K .

Because $\oplus$ is its own inverse, $\{M = m, C = c\}$ is the same event as $\{M = m, K = m \oplus c\}$. So the whole joint law is

$$\mathbb{P}(M = m, C = c) = \mathbb{P}(M = m) \mathbb{P}(K = m \oplus c),$$

and by the law of total probability

$$\mathbb{P}(C = c) = \sum_m \mathbb{P}(M = m) \mathbb{P}(K = m \oplus c).$$

Everything I compute about the cipher comes from these two lines.

Model II: the distributions I use

I take $n = 2$, so there are four messages, four keys, four ciphertexts, and every answer can be computed exactly.

Variable	Distribution	Role
M	$\mathbb{P}(00, 01, 10, 11) = (0.4, 0.3, 0.2, 0.1); H(M) = 1.85 \text{ bits}$	lopsided on purpose
K	independent of M ; one of	
A	$\text{Unif}(\{0, 1\}^2); H(K) = 2 \text{ bits}$	one-time pad
B	$K_1, K_2 \text{ i.i.d. Bern}(q), q \in [0.5, 0.95]; H(K) = 2h(q)$	the sweep
C	$C = M \oplus K; \mathbb{P}(C = c) = \sum_m \mathbb{P}(M = m) \mathbb{P}(K = m \oplus c)$	what the attacker sees

The message is lopsided so that there is something to leak. If M were uniform, every cipher would look perfect.

Model II: what “secure” means

Shannon (1949): a cipher is **perfectly secret** if for every m and c

$$\mathbb{P}(M = m \mid C = c) = \mathbb{P}(M = m).$$

Seeing the ciphertext does not change what you believe about the message.

Model II: what “secure” means

Shannon (1949): a cipher is **perfectly secret** if for every m and c

$$\mathbb{P}(M = m \mid C = c) = \mathbb{P}(M = m).$$

Seeing the ciphertext does not change what you believe about the message.

The one-time pad has it. With K uniform on $\{0, 1\}^n$,

$$\mathbb{P}(C = c \mid M = m) = \mathbb{P}(K = m \oplus c) = 2^{-n},$$

which does not depend on m . Total probability gives $\mathbb{P}(C = c) = 2^{-n}$ too, and Bayes' rule gives

$$\mathbb{P}(M = m \mid C = c) = \frac{2^{-n} \mathbb{P}(M = m)}{2^{-n}} = \mathbb{P}(M = m).$$

Model II: what “secure” means

Shannon (1949): a cipher is **perfectly secret** if for every m and c

$$\mathbb{P}(M = m \mid C = c) = \mathbb{P}(M = m).$$

Seeing the ciphertext does not change what you believe about the message.

The one-time pad has it. With K uniform on $\{0, 1\}^n$,

$$\mathbb{P}(C = c \mid M = m) = \mathbb{P}(K = m \oplus c) = 2^{-n},$$

which does not depend on m . Total probability gives $\mathbb{P}(C = c) = 2^{-n}$ too, and Bayes' rule gives

$$\mathbb{P}(M = m \mid C = c) = \frac{2^{-n} \mathbb{P}(M = m)}{2^{-n}} = \mathbb{P}(M = m).$$

Any other key breaks it. If $\mathbb{P}(K = k)$ varies with k , then $\mathbb{P}(C = c \mid M = m)$ varies with m , and the ciphertext carries information. The next slide says how much.

Model II: measuring the leak

Uncertainty before and after seeing the ciphertext:

$$H(M) = - \sum_m \mathbb{P}(M = m) \log_2 \mathbb{P}(M = m), \quad H(M | C) = \sum_c \mathbb{P}(C = c) H(M | C = c).$$

The difference is the mutual information, the number of bits that leaked:

$$I(M; C) = H(M) - H(M | C) \geq 0.$$

Perfect secrecy is the same thing as $I(M; C) = 0$.

Model II: measuring the leak

Uncertainty before and after seeing the ciphertext:

$$H(M) = - \sum_m \mathbb{P}(M = m) \log_2 \mathbb{P}(M = m), \quad H(M | C) = \sum_c \mathbb{P}(C = c) H(M | C = c).$$

The difference is the mutual information, the number of bits that leaked:

$$I(M; C) = H(M) - H(M | C) \geq 0.$$

Perfect secrecy is the same thing as $I(M; C) = 0$.

For this cipher there is a shortcut. Given M , the ciphertext is a shifted copy of the key, so $H(C | M) = H(K)$ and

$$I(M; C) = H(C) - H(K) \geq H(M) - H(K).$$

The bound is Shannon's: a key with less entropy than the message must leak at least the difference. This is the curve I will test.

Study 2: measure the ciphers

Caesar. Message is English text, key is a shift $K \sim \text{Unif}\{0, \dots, 25\}$, so $H(K) = \log_2 26 = 4.7$ bits. Shannon's bound says $I(M; C) \geq H(M) - 4.7$ bits. For a paragraph of text, $H(M)$ is hundreds of bits. Nearly all of it leaks. The letter histogram of the ciphertext is the plaintext's histogram, shifted.

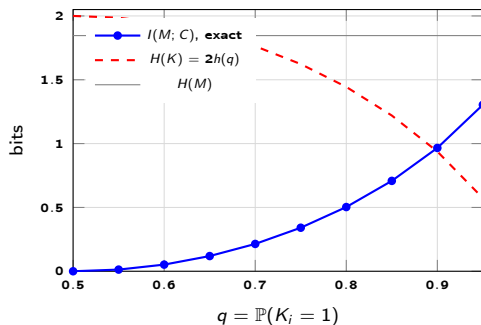
One-time pad. Key from source A. Exact leakage 0.

Method. Draw $N = 10^4$ pairs (M, C) , build the 4×4 table of counts, plug the frequencies into $H(M) - H(M | C)$. Compare with the exact value.

Something I already know will happen. An estimate from a finite sample is always a little above zero, even for the one-time pad. I will show the estimate falling towards zero as N grows, rather than hide it.

Study 3: the main experiment

Fix the cipher. Fix the message. Only change the key source, with K_1, K_2 i.i.d. Bern(q) for $q = 0.50, 0.55, \dots, 0.95$.



The exact curve, computed today from $I(M; C) = H(C) - H(K)$.

At $q = 0.5$ the key is a one-time pad and nothing leaks. At $q = 0.95$ the key has 0.57 bits of entropy and 1.30 of the message's 1.85 bits leak.

Method. At each q , draw $N = 10^4$ pairs, estimate $I(M; C)$ as in Study 2, repeat 50 times for error bars. Plot the estimates on top of this curve, and again against $H(K)$.

Wrong if the simulated points miss the curve by more than the error bars. That would mean a bug in the estimator or in my derivation, and either is worth finding.

What I expect, and what would surprise me

Expected

- Human sequences fail the runs test, pass the frequency test.
- Caesar leaks nearly all of $H(M)$.
- One-time pad: $I(M; C) \rightarrow 0$ as N grows.
- Leakage rises as $H(K)$ falls, never below $H(M) - H(K)$.
- The LCG passes all three tests and is predictable from a handful of outputs.

Would surprise me

- People who pass the runs test. Then I would look at the length of the longest run instead.
- A simulated point below Shannon's bound. Impossible if the code is right, so it is a bug detector.
- A gap between simulation and the exact curve that does not shrink like $1/N$.

Every claim comes with a way to fail. That is what makes this a project and not a demonstration.

Plan and deliverables

Week	Work
1	Sources and the three tests. Calibrate on a fair coin.
2	Collect human sequences. Fit s . Write Part I.
3	Caesar, XOR, one-time pad. Entropy estimators. Part II.
4	Main experiment. Exact curve. Main figure.
5	Recover the LCG state. Interactive program.
6	Introduction and conclusion last. Proof-read.

Deliverables

- The report, as a PDF in the GitHub repository (paper/main.pdf).
- The code, in the same repository (code/).
- A small program: type in a sequence and see all three tests run on it; encrypt a message and see what leaks.

Simulations before writing in every part.
The report describes results I have, not results I expect.

Takeaway

Good randomness is necessary for secrecy.

Looking random is not enough to make something secure.

Statistics tests the output. Security is about what the attacker knows. The project measures the distance between the two.